





Chapter 1: Introduction to Compilers

The Translator's Journey: From Source Code to Machine Code

By
Dr. Adnan M. Elsherif, PhD

CS432 – Fall 2026/2027

1

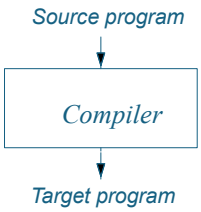
Agenda

- What is a Compiler?
- Compilers vs. Interpreters
- The Structure of a Compiler: Phases Overview
- Deep Dive into Key Phases (with Example)
- Compiler Construction Tools
- Evolution of Programming Languages
- Summary

2

What is a Compiler?

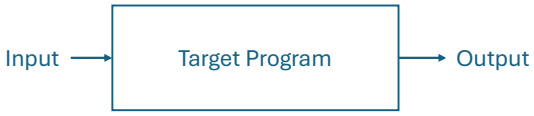
- **Definition:** A program that translates code from a **source language** (e.g., C++, Java) into an equivalent program in a **target language** (e.g., machine code).
- **Key Role:** Reports errors detected during translation.



3

Compiler vs. Interpreter

- **Compiler:** Translates entire program upfront. Target program is then executed.
  - *Advantage:* Execution is very fast.



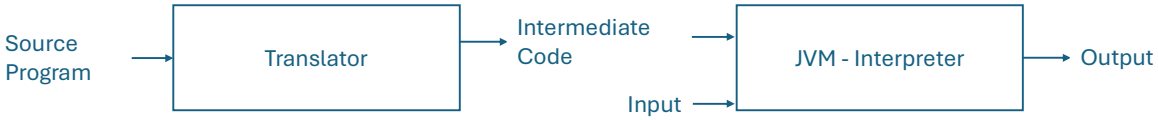
- **Interpreter:** Directly executes the source program statement by statement.
  - *Advantage:* Better error diagnostics and debugging.



4

Hybrid Approach: The Java Example

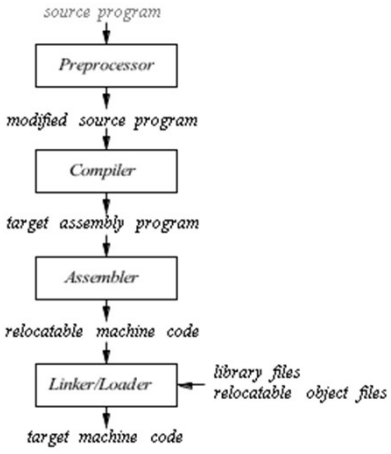
- Many modern systems (like Java) use both compilation and interpretation.
- **Process:**
  - Source code is compiled into intermediate **bytecodes**.
  - Bytecodes are interpreted by a **Virtual Machine (VM)**.
- **Just-in-Time (JIT) Compilers:** Translate bytecodes to machine code at runtime for speed.



5

The Full Language Processing System

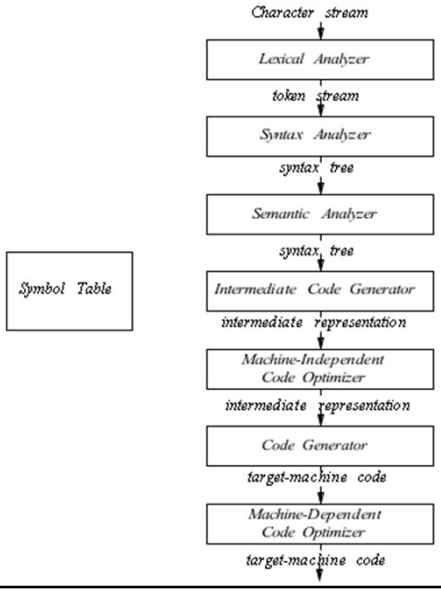
- A compiler is often part of a larger toolchain.
  - **Preprocessor:** Handles macros and file inclusion.
  - **Compiler:** Generates assembly code.
  - **Assembler:** Converts assembly to relocatable machine code.
  - **Linker/Loader:** Combines multiple object files and libraries into an executable.



6

The Structure of a Compiler: Two Major Parts

- **Front End (Analysis):**
  - Language-dependent.
  - Breaks down source code, checks for correctness (syntax/semantics).
  - Output: Intermediate Representation (IR) + Symbol Table.
- **Back End (Synthesis):**
  - Machine-dependent.
  - Takes IR and generates target machine code.
  - Often includes optimization.



7

Running Example

- Source Code:

*position = initial + rate * 60*

- We will follow this statement through the compilation phases.

8

Phase 1 - Lexical Analysis (Scanning)

- Task: Read character stream, group into meaningful lexemes, output tokens.
- Token Format: *<token-name, attribute-value>*
- Example Output for our statement:

*position = initial + rate * 60*

<id, 1> <=> <id, 2> <+> <id, 3> <*> <number,60>

This is accomplished using Regular Expressions
id: [a-zA-Z_][a-zA-Z0-9_]*
number: [0-9]+

Symbol Table

Seq.	Lexeme	Attributes
1	*position*	null
2	*initial*	null
3	*rate*	null

9

Phase 2 - Syntax Analysis (Parsing)

- Task: Build a syntax tree that represents grammatical structure. To accomplish this task a Context free Grammer of the language is provided.
- For our example we assume the following CFG:

<assign> -> id = <expr>
<expr> -> <expr> + <term> | <term>
<term> -> <term> * <factor> | <factor>
<factor> -> (<expr>) | id | number

- Activity: Create parse tree for the following sequence of tokens

<id, 1> <=> <id, 2> <+> <id, 3> <*> <number,60>

10

Phase 3 - Semantic Analysis

- Task: Check for semantic soundness (e.g., type checking).
- Example: If rate is float and 60 is integer, the compiler inserts a type conversion.

11

Phase 4 - Intermediate Code Generation

- Task: Generate machine-like, optimizable code (Three-Address Code).
- Example Output:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

12

Phase 5 - Code Optimization

- Task: Improve code to be faster/smaller.
- Example: Do the conversion at compile time! Replace 60 with 60.0.

Before optimization	After optimization
t1 = inttofloat(60) t2 = id3 * t1 t3 = id2 + t2 id1 = t3	t1 = id3 * 60.0 id1 = id2+t1

13

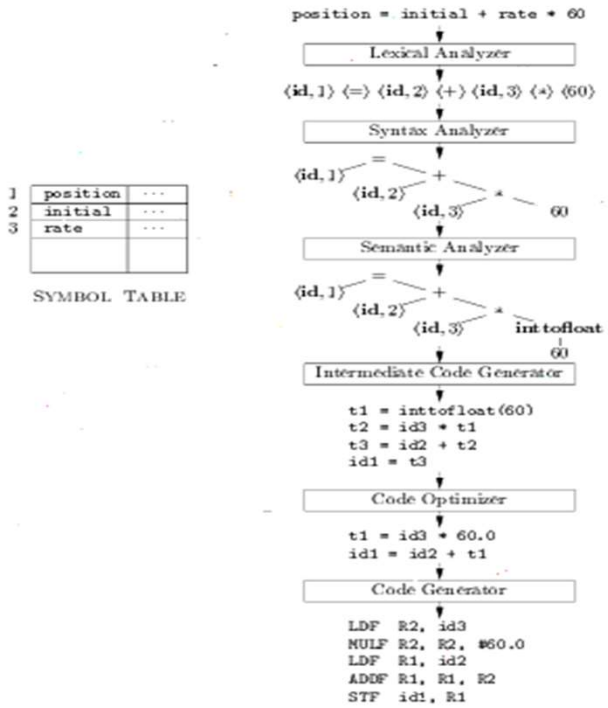
Phase 6 - Code Generation

- Task: Map to target machine code. Handle registers and instructions.
- Final Output (Example):

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

14

The Complete Picture



15

Compiler Construction Tools

- Tools that automate compiler creation:
  - **Parser Generators** (e.g., Yacc, Bison)
  - **Scanner Generators** (e.g., Lex, Flex)
  - **Syntax-Directed Translation Engines**
  - **Code-Generator Generators**
  - **Data-Flow Analysis Engines**

16

Evolution of Programming Languages

- **1st Gen:** Machine Code (0s and 1s)
- **2nd Gen:** Assembly Language (Mnemonics)
- **3rd Gen:** High-Level Languages (C++, Java, Python)
- **4th Gen:** Domain-Specific Languages (SQL)
- **5th Gen:** Logic-based Languages (Prolog)

17

Summary

- Compilers are essential translators.
- Divided into a **Front End** (analysis) and **Back End** (synthesis).
- Key phases: Lexical, Syntax, Semantic, Intermediate Code, Optimization, Code Generation.
- The **Symbol Table** is a critical data structure.

18